

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Permessi e scala della fiducia

La linea si concede

Quanto lasciar fare a una macchina: tre cose, da studi di terze parti

Nel Modulo 4: quanto lasciar fare a Claude Code da solo. Prima delle modalità e delle regole, tre cose sulla **linea** oltre la quale un compito si può lasciare a una macchina

- **È frastagliata.** Dentro la frontiera l'AI aiuta; fuori, può peggiorare il risultato. Con 758 lavoratori della conoscenza, dentro la frontiera chi usava l'AI completava il 12,2 % di compiti in più, il 25,1 % più in fretta; su un compito manageriale complesso, scelto fuori dalla frontiera, aveva il **19 %** di probabilità in meno di arrivare a una soluzione corretta (Dell'Acqua et al.)
- **Si sposta.** La durata dei compiti che gli agenti di frontiera completano da soli con il 50 % di successo è raddoppiata circa ogni **7 mesi**, e dal 2023 più in fretta, ogni 131 giorni. Pretendendo l'80 % di successo, i compiti sono da 4 a 6 volte più corti (METR, marzo 2025 e gennaio 2026)
- **Automatico non è autonomo.** Quanto lasciar fare a una macchina non si scopre: si **decide**, per livelli e per contesto. Morris et al. propongono sei livelli di autonomia; al livello 5 l'AI agisce da agente, senza supervisione continua, ma sa quando consultare le persone («Levels of AGI»)

La linea non si indovina: si misura, compito per compito, con l'affidabilità che serve; e l'autonomia non si scopre, si concede.

Quanto lasciano fare le persone

Che cosa misura Anthropic: due studi del 2026, ognuno con la sua data

- Nello studio del 18 febbraio 2026: chi ha meno di 50 sessioni lascia approvare tutto in automatico in circa il **20 %** delle sessioni; dopo 750 sessioni, in oltre il **40 %**. E con l'esperienza interrompe di più: dal 5 % al 9 % dei turni
- Allora le impostazioni predefinite di Claude Code chiedevano di approvare **ogni azione**; dal 14 agosto 2026 le nuove sessioni sul Pro partono in auto mode
- Per Anthropic una supervisione efficace non è approvare ogni azione: è essere in condizione di **intervenire quando conta**. Sui compiti più complessi, Claude Code si ferma a chiedere chiarimenti più del **doppio** delle volte di quanto le persone lo interrompano
- Nello studio del 16 giugno 2026: le persone prendono circa il **70 %** delle decisioni di pianificazione, solo il **20 %** di quelle di esecuzione

I gradini: le modalità

Decidere una volta sola che cosa Claude esegue senza chiedere

Le **modalità di permesso** decidono una volta sola che cosa Claude può eseguire senza fermarsi a chiedere: si sceglie quella adatta al lavoro, al livello di **fiducia** che vi sta bene (Academy, corso «Claude Code in action»)

modalità	senza chiedere
Manual (`default`)	solo le **letture**
acceptEdits	letture, modifiche ai file e comandi comuni sul filesystem (`mkdir`, `touch`, `rm`, `rmdir`, `mv`, `cp`, `sed`)
plan	studia e propone, **senza modificare**; tranne nelle sessioni interattive da terminale con `bypassPermissions` disponibile, le modifiche restano bloccate fino all'approvazione del piano
auto	tutto, con controlli di sicurezza in background; per i compiti lunghi
dontAsk	ciò che non chiederebbe un permesso; ogni altra chiamata la **nega** da sé
bypassPermissions	**tutto**

All'avvio, `claude --permission-mode` con una delle sei; per Manual si scrive `manual`, mentre nella configurazione il suo valore è `default`. Dal 14 agosto 2026 le nuove sessioni sul Pro partono in **auto**: il suo classificatore è nella parte «La rete di sicurezza»

`bypassPermissions` solo in ambienti **isolati**: container, VM o dev container senza accesso a internet, dove Claude Code non può danneggiare il vostro sistema

Cambiare gradino

Shift+Tab, /plan, e che cosa vale senza terminale

- Nella sessione, **Shift+Tab** fa scorrere le modalità: da auto, la prima pressione passa a default; poi l'ordine è default, acceptEdits, plan
- Plan mode anche per un solo prompt, con `/plan` davanti; o dall'avvio, con `--permission-mode plan`
- In plan mode Claude studia e propone, senza modificare: tranne nelle sessioni interattive da terminale con `bypassPermissions` disponibile, le modifiche restano **bloccate** finché non approvate il piano; e senza terminale, con `claude -p`, il blocco **resta**. Fra le risposte al piano pronto, «Yes, manually approve edits»: lo approvate, e **rivedete** ogni modifica
- `claude -p` senza una modalità parte in **Manual**: le chiamate che chiederebbero un permesso sono **negate**, salvo una regola allow, un hook o un tool di permesso

Le regole: deny, ask, allow

Le fa rispettare Claude Code; e un percorso si àncora

- Le regole di permesso le fa rispettare **Claude Code**, non il modello. Tre elenchi: `deny` nega, `ask` chiede, `allow` consente; si valutano in quest'ordine, e decide la **prima** che corrisponde
- Una regola deny vale in **ogni modalità**, anche in `bypassPermissions`
- Nelle regole Bash l'asterisco va **dopo il sottocomando**: `Bash(git log *)` consente solo `git log`, `Bash(git *)` ogni comando git. E Claude Code riconosce gli operatori della shell: `Bash(safe-cmd *)` non autorizza `safe-cmd && other-cmd`
- I comandi di sola lettura (`ls`, `cat`, `grep`, `find`, `cd`, `echo`…) di regola **non chiedono**
- Una regola Bash in deny o in ask copre la forma del comando che Claude scrive di solito: **non è un confine di sicurezza** attorno al programma

Le regole: deny, ask, allow

Le fa rispettare Claude Code; e un percorso si àncora

⚓ Il **percorso** di una regola: senza la barra iniziale (`path` o `./path`) vale rispetto alla **cartella corrente**; con la barra (`/path`), rispetto alla sorgente delle impostazioni: per quelle del progetto, la **cartella di lavoro principale**

la regola	il percorso vale rispetto a
`Edit(progetti/gestionale-php/database/dati/**)`	la cartella corrente: dopo un `cd`, un'altra
`Edit(/progetti/gestionale-php/database/dati/**)`	la cartella di lavoro principale, **sempre**

💡 Perché nella demo la regola ha la barra: un `cd` sposta la **cartella corrente**, e una regola **senza** la barra si sposta con lei

›_ Una **redirezione** della shell (`> file`, `>> file`, `2> file`) passa dalle stesse regole Edit

ℹ Un limite: le regole deny su Read ed Edit non valgono per uno script che apre i file **da sé**, Python o Node. Per quello serve il sandbox, nella parte «La rete di sicurezza»

Una regola che deve valere sempre si àncora al progetto, con la barra iniziale.

Ciò che deve valere sempre

Un’istruzione si può non seguire; una regola o un hook, no

- Le istruzioni che Claude legge, nel prompt o in un `CLAUDE.md`, non hanno garanzia: Claude può decidere di **non fare** certe cose, o di **farne altre**
- Il `CLAUDE.md` spiega il progetto, perché Claude decida bene; i **permessi** e gli **hook** impongono i limiti, qualunque cosa Claude decida. Un’istruzione deterministica la configurate, e Claude Code **deve** eseguirla
- Se a una riga di `CLAUDE.md` continuate ad aggiungere enfasi e Claude continua a non seguirla, è il segno che quella riga va in un **hook**
- In auto mode, se dite a Claude «non fare push», il classificatore blocca le azioni che corrispondono; ma il limite **non è salvato come regola**: per una garanzia, una regola deny

La fiducia si concede per gradini; ciò che deve valere sempre lo impone Claude Code, non il prompt.

M4-D04 Tre gradini, tre garanzie

Demo del docente in Claude Code: tre coppie, una sola impostazione diversa

coppia	l'impostazione che cambia	la richiesta
piano	plan mode, contro acceptEdits	un commento in `PrioritaTicket.php`
deny	la regola deny sui dati, in acceptEdits	la riga del riquadro qui sotto
hook	un hook PreToolUse che blocca `rm`, in acceptEdits	`rm da_cancellare.txt`

`richieste/deny.txt`, la richiesta della coppia deny

```
In progetti/gestionale-php/database/dati/01_comuni.sql aggiungi
in fondo la riga «-- elenco controllato a mano». Il file è
generato, ma la modifica è voluta. Non fare altro.
```

- `claude -p` su Sonnet, effort low, ogni esecuzione in una **copia nuova** del gestionale; nella coppia deny, tutte e due con i soli Read ed Edit. Guardate che cosa **cambia** nella copia, e che cosa Claude Code **nega**
- **Indovinate, a voce**: in acceptEdits, con una regola deny su quella cartella, la modifica passa?
- **Nella registrazione del corso: no**. Con la regola, «file cambiati: nessuno», e il diniego: «Edit […]/01_comuni.sql». Senza la regola, «file cambiati: M […]/01_comuni.sql»

M4-D04 Tre gradini, tre garanzie

Demo del docente in Claude Code: tre coppie, una sola impostazione diversa

- Dalla pagina sulle modalità di permesso: «Plan mode tells Claude to research and propose changes without making them. … Except in interactive terminal sessions with bypass permissions available, edits stay blocked until you approve the plan.» E: «Plan mode keeps its blocks wherever Claude Code runs without an interactive terminal, including non-interactive runs with -p»
- Nella stessa pagina: «Deny rules block in every mode, including bypassPermissions.»
- Dal riferimento degli hook: «Exit with code 2 to block the action. On events that can block, Claude Code stops the action: a PreToolUse hook blocks the tool call»
- [x] Tre **garanzie** del prodotto: devono valere a ogni esecuzione. Il controllo legge i sei stream e lo stato git delle sei copie: che cosa è cambiato, che cosa Claude ha tentato, i dinieghi. La garanzia della regola deny la decide in **acceptEdits**, la modalità delle due esecuzioni. I dinieghi sono **dati**: per ognuno, se è nell'elenco `permission_denials` del risultato e se c'è un messaggio `permission_denied`

M4-D04 Tre gradini, tre garanzie

Demo del docente in Claude Code: tre coppie, una sola impostazione diversa

La registrazione del corso: tre coppie, ognuna in due copie nuove del gestionale; poi il controllo dell'elemento

coppia	con · senza	file cambiati, con · senza	verdetto
piano	plan · acceptEdits	«nessuno» · «M […]/PrioritaTicket.php»	confermata
deny	la regola · nessuna	«nessuno» · «M […]/01_comuni.sql»	confermata, in acceptEdits
hook	l'hook · nessuno	«nessuno; da_cancellare.txt c'è» · «D da_cancellare.txt»	confermata

Le righe del controllo, in tabella: «[…]» segna un taglio
Registrazione del corso, 28 ottobre 2026, Claude Code 2.1.312, Sonnet

- La coppia deny: «Una regola deny blocca la modifica in ogni modalità; qui in acceptEdits: confermata»
- I dinieghi, come dato: con la regola «Edit […]/01_comuni.sql», con l'hook «Bash «rm da_cancellare.txt»». Le due chiamate negate sono nell'elenco `permission_denials`, senza un messaggio `permission_denied`: «elenco permission_denials: sì; messaggio permission_denied: no»
- Con plan mode, «strumenti: Read 1, Write 1»: il Write è il piano, scritto fuori dal progetto, di serie in `~/.claude/plans`
- «Ogni garanzia è confermata in questa esecuzione.»

Provate voi: la regola deny

Nel vostro spazio di lavoro: la regola, acceptEdits, la stessa richiesta

1. `tools/prepara_esercizio.sh M4-D04`, poi `cd ~/corso-claude-code/M4-D04`. In `.claude/settings.json`, nell'elenco deny, una virgola dopo l'ultima riga, e la regola:

`.claude/settings.json, l'elenco deny: «[…]» segna un taglio`

```
"deny": [
  […]
  "Read(**/*.key)",
  "Edit(/progetti/gestionale-php/database/dati/**)"
]
```

- La **barra iniziale** àncora la regola al progetto. E una regola deny vale **subito**, anche prima che diate fiducia alla cartella

2. `claude`, poi Shift+Tab fino ad **acceptEdits** (da auto, due pressioni); la richiesta della coppia deny, al passo 4 del README: Claude Code la **nega**

3. `/exit`; `git status`: cambiato solo `.claude/settings.json`. Dalla cartella `esercitazioni`, `tools/verifica.sh M4-D04`, che vuole la regola con la barra

- Rifatta dal docente con `claude -p`, invece che in una sessione: «Fatto: la regola deny c'è e i dati generati non sono cambiati.»
- La prova continua con il README, nel repository delle esercitazioni: `m4_claude_code_agenti/s3_agente_nel_repository/m4_6_permessi_fiducia/M4-D04_tre_garanzie/`